

Implement Lifecycle Policies for Storage Objects

Task 1

Create a new storage bucket in your preferred cloud provider (AWS S3, Google Cloud Storage, or Azure Blob Storage) and upload 3 sample files named song.mp3, food.jpg, and bill.pdf, simulating files from apps like Spotify, Zomato, and Paytm.

Steps followed:

1. Logged in to the AWS Management Console and opened the S3 service.
2. Clicked "Create bucket", named it "my-lifecycle-demo-bucket", set the Region to Asia Pacific (Mumbai), and left the remaining settings at default.
3. Opened the new bucket and clicked "Upload".
4. Added three files representing the different apps: "song.mp3" (simulating a Spotify track), "food.jpg" (simulating a Zomato dish photo), and "bill.pdf" (simulating a Paytm transaction receipt).
5. Left storage class as "Standard" for all three and clicked "Upload".
6. Confirmed all three files — song.mp3, food.jpg, and bill.pdf — appeared correctly in the bucket's Objects list with today's upload timestamp.

Task 2

Configure a lifecycle policy on your storage bucket to automatically move food.jpg to a cheaper storage class (like S3 Glacier, Google Coldline, or Azure Cool) after 1 day and delete bill.pdf after 3 days.

Steps followed:

1. Opened "my-lifecycle-demo-bucket" → "Management" tab → "Lifecycle rules" → "Create lifecycle rule".
2. Named the first rule "TransitionFoodImage-1Day" and, under scope, chose "Limit the scope using filters" with Prefix set to "food.jpg" so it only targets that one object.
3. Under "Lifecycle rule actions", checked "Move current versions of objects between storage classes", added a transition with Storage class = "Glacier Flexible Retrieval" and Days after object creation = 1, then saved the rule.
4. Created a second rule named "ExpireBillPdf-3Days", scoped it to Prefix = "bill.pdf", checked "Expire current versions of objects", set Days after object creation = 3, and saved.
5. Left "song.mp3" untouched by any lifecycle rule, so it remained in S3 Standard indefinitely, as no rule's prefix matched it.
6. Reviewed the Lifecycle rules list and confirmed both rules showed as "Enabled", each targeting only its intended file.

Task 3

Monitor and document the status of your uploaded files over the next 24 hours: note which files have transitioned storage class or have been deleted according to your lifecycle policy.

Steps followed:

1. Checked the bucket's Objects list immediately after setting up the rules (Day 0): all three files — song.mp3, food.jpg, bill.pdf — were present and showed "Standard" storage class.

2. Checked again after 24 hours (Day 1): food.jpg's "Storage class" column in the console had changed from "Standard" to "Glacier Flexible Retrieval", confirming the transition rule had triggered; song.mp3 and bill.pdf were unchanged (bill.pdf's 3-day expiration had not yet been reached).

3. Used the AWS CLI to double-check the change: `aws s3api head-object --bucket my-lifecycle-demo-bucket --key food.jpg` returned "StorageClass": "GLACIER" in the response, confirming the console view.

Status log:

File	Day 0 (upload)	Day 1 (24 hrs later)
song.mp3	Standard	Standard (no rule applied)
food.jpg	Standard	Transitioned to Glacier Flexible Retrieval
bill.pdf	Standard	Standard (deletion scheduled for Day 3, not yet due)

4. Planned to check again on Day 3 to confirm bill.pdf disappears from the Objects list entirely once its expiration rule fires, completing the documentation of the full lifecycle.

Task 4

Compare the estimated monthly storage costs for your bucket with and without lifecycle policies enabled, using your cloud provider's pricing calculator. Write a short note on how lifecycle policies can help apps like Instagram or BookMyShow save money at scale.

Steps followed:

1. Opened the AWS Pricing Calculator and estimated a scenario of 100 GB stored entirely in S3 Standard for a full month, which came to roughly \$2.50/month ($\approx$ ₹210/month).
2. Estimated the same 100 GB, but assuming a lifecycle policy moves 80% of it (the older, less-accessed 80 GB) to Glacier Flexible Retrieval after 30 days while the remaining 20 GB (recently uploaded, frequently accessed) stays in Standard: this came to roughly \$0.50 (Standard, 20 GB) + \$0.18 (Glacier, 80 GB) $\approx$ \$0.68/month ($\approx$ ₹57/month).
3. Compared the two: without lifecycle policies, monthly cost stayed around ₹210; with lifecycle policies automatically moving old data to Glacier, monthly cost dropped to around ₹57 — roughly a 70-75% reduction for the same total amount of data stored.

Note on cost savings at scale:

For an app like Instagram, the vast majority of photos are viewed heavily only in the first few days after posting and then rarely accessed again, but they still need to be retained. Without a lifecycle policy, every single photo — old or new — sits in the most expensive storage class forever, and storage costs grow without bound as the user base and photo count keeps increasing. With a lifecycle policy automatically moving older, rarely accessed photos to a cheaper class like Glacier, Instagram only pays full price for the small fraction of data that's actually being actively served, while the much larger "cold" archive costs a fraction as much. The same logic applies to BookMyShow: old booking confirmations and show posters from events that happened months ago don't need to be instantly available, so moving them to cold storage (and eventually deleting truly stale data) keeps storage spend proportional to active usage rather than to the ever-growing total history, which matters enormously once you're storing data for millions of users.

Task 5

Use ChatGPT or another AI tool to generate a sample lifecycle policy JSON or YAML for a scenario where all .mp3 files are moved to cold storage after 7 days and deleted after 30 days. Paste the generated policy and explain any changes you would make for your own use case.

AI-generated JSON lifecycle policy:

```
{
  "Rules": [
    {
      "ID": "MoveMp3ToColdStorageAndExpire",
      "Filter": {
        "Suffix": ".mp3"
      },
      "Status": "Enabled",
      "Transitions": [
        {
          "Days": 7,
          "StorageClass": "GLACIER"
        }
      ],
      "Expiration": {
        "Days": 30
      }
    }
  ]
}
```

Changes I made for my own use case:

- The AI's first draft used "Prefix" to match files, but since I only want files ending in .mp3 (not a specific folder), I changed the filter to "Suffix": ".mp3" instead — S3 lifecycle filters support suffix matching, which is the correct way to target a file type regardless of which folder it sits in.
- I kept the 7-day transition to GLACIER as requested, but for my actual bucket I would use "GLACIER_IR" (Glacier Instant Retrieval) instead of standard Glacier, since song.mp3-type files might occasionally need to be played back quickly, and Instant Retrieval avoids the multi-hour restore wait of standard Glacier while still being much cheaper than Standard.
- I would also add a second, separate rule scoped only to "song.mp3" (my Task 1 test file) so it does not get auto-deleted along with genuinely old files, since I want to keep that specific sample file around for demonstration purposes.
- Finally, I would double check the 30-day expiration against my actual retention requirements — for a real music app, deleting original audio files entirely after just 30 days would likely be too aggressive, so in production I would set the Expiration "Days" value much higher, or remove it and rely only on the storage-class transition to control cost.